# Battery savings analysis — design

Status: **implementation in progress.** `prices.py` is built and validated against the live API; `pricing.py` is built and pending validation against real `power_readings`. The Grafana dashboard and daily scheduling are still to do. This document remains the spec.

## Goal

Quantify what the home battery is worth in euros, by pricing the same measured household against two worlds:

- **Model 1 — with battery (actual).** Price the energy that actually flowed through the grid meter. Used to **validate** our accounting against the real Frank Energie bill.
- **Model 2 — without battery (counterfactual).** Assume the battery was never installed; whatever it charged would have been exported, whatever it discharged would have been imported. Price that.

**Battery value = cost(Model 2)** – **cost(Model 1)**, per day and over any selected period. A day can show a *loss* (round-trip losses / poor timing outweigh arbitrage); the sign is kept.

## Data we have

Measurement `power_readings` in bucket `alphaess`, tag `sys_sn`, sampled every 30 s by `collector.py`:

| Field | Unit | Sign convention |
|---|---|---|
| `pv_power_w` | W | solar generation (≥0) |
| `grid_power_w` | W | **+ = import,** – **= export** |
| `load_power_w` | W | house load (≥0) |
| `battery_power_w` | W | **+ = discharge,** – **= charge** |
| `soc_percent` | % | battery state of charge |

Sign conventions are the AlphaESS API defaults (`pgrid`, `pbat`); **verify with `collector.py --once` before trusting any euro figure** — every result below inverts if a sign is flipped.

## The accounting (why the counterfactual is nearly free)

At the AC bus, every instant obeys:

```
load = pv + grid + battery          (grid: +import/−export, battery: +discharge/−charge)
```

Remove the battery (`battery := 0`) with `load` and `pv` unchanged:

```
grid_cf = load − pv = grid_actual + battery_power_w
```

So the no-battery grid series is just **the actual grid plus the battery power added back**. Both models then run through the *same* pricing engine; only the grid series differs.

Consequences worth stating:

- **Round-trip efficiency is already captured.** `battery_power_w` is measured at the AC bus, so charge-Wh naturally exceed discharge-Wh over time. The counterfactual correctly credits *not* wasting that ~10–15% loss. No efficiency fudge factor.
- **Grid arbitrage is captured for free.** If the battery grid-charges on cheap intervals and discharges on expensive ones, that shows up automatically in `grid_cf` vs `grid_actual`.
- **Balance closure is a data-quality gate.** Model 2 is only as true as `pv + grid + battery ≈ load` in the hardware. We compute the daily residual and store it; large-residual days are flagged/excluded (also catches sign mistakes early).

## Pricing model

### Atomic slot = Frank's price interval

Frank Energie bills at **hourly** granularity in 2026 (the hourly price is the average of the four 15-min EPEX values). We do **not** hardcode 15 vs 60 min: `prices.py` stores whatever `from`/`till` intervals the API returns, and `pricing.py` integrates power within those exact boundaries. If Frank moves to 15-min settlement later, the model adapts with no code change.

### Per-slot netting is *exact* for 2026 (not an approximation)

Frank confirmed that **before 2027 saldering nets in full** — energy tax and BTW are refunded on returned electricity. Combined with our assumption that **annual export** ≤ **import** (we have <1 year of data, so we net everything):

- **Commodity (EPEX ± markup):** a dynamic contract already prices each interval at its own rate. Per-slot *is* the mechanism. Exact.
- **Energy tax + BTW:** the rate is *flat* across slots, so `Σ (import_i − export_i) × tax = (Σimport − Σexport) × tax`. Per-slot netting equals the legally-correct annual volume netting. Exact.

Therefore daily results are **additive** — a day is self-contained *and* the annual total is right. Period stats are plain sums of daily rows. (This exactness **ends in 2027** when tax netting on export is removed — see Open items.)

### Prices come from Frank's API, per component

`marketPricesElectricity` (endpoint `https://frank-graphql-prod.graphcdn.app/`, public GraphQL, no auth, confirmed live) returns per interval:

```
total = marketPrice + marketPriceTax + sourcingMarkupPrice + energyTaxPrice
```

The components are BTW-handled per-part: `marketPriceTax` is the 21% BTW on the market price, and `sourcingMarkupPrice` / `energyTaxPrice` are themselves BTW-inclusive. So `total` is the fully all-in consumption price (€/kWh). One call returns one Amsterdam local day (23/24/25 hourly rows across DST).

We store all four components + the all-in `total` + `from`/`till`. This means:

- Model 1 matches the bill **to the cent** using Frank's own numbers.
- Rate changes (annual energy-tax updates, the 2027 saldering cliff) are tracked automatically — no hardcoded €0.0175 markup or €0.09161 tax.
- Every euro is decomposable for audit.

Reference figures for 2026 (informational — the API is the source of truth; live values observed on 2026-07-18): `sourcingMarkupPrice` ≈ €0.01815/kWh (incl. BTW), `energyTaxPrice` ≈ €0.11085/kWh (incl. BTW, = €0.09161 excl. × 1.21); BTW 21%; fixed delivery €4.99/mo; tax credit €628.96/yr (fixed costs cancel in the Model 2 − Model 1 difference).

### Import vs export price

Per slot *i*, with `import_i`/`export_i` the integrated grid energy (kWh):

```
cost_actual_i = import_actual_i · p_import_i − export_actual_i · p_export_i
cost_cf_i     = import_cf_i     · p_import_i − export_cf_i     · p_export_i
saving_i      = cost_cf_i − cost_actual_i
```

- `p_import_i` = Frank's all-in `total` for the slot.
- `p_export_i` (salded, 2026) — **implemented as option (b)**; still to be pinned against a real teruglevering bill line after 2026-07-26:
- (a) Frank's `feedIn` price field directly, **or**
- (b) **(implemented)** `marketPrice + marketPriceTax − sourcingMarkupPrice + energyTaxPrice` — commodity credited per-slot with the markup deducted, energy tax refunded under saldering, BTW kept.
- We **exclude the ~15% teruglever bonus** (it applies to specific cases only), which is why option (b) is the default — it keeps the bonus out.

### The surprise this model will show

The flat energy-tax refund is decoupled from the slot price and, under 2026 saldering, is refunded on export. So even at a **negative** EPEX slot, exported energy is still worth **positive** money (tax refund dominates). Saldering already rescues midday exports, so the battery's 2026 benefit is **much smaller** than a raw-EPEX view suggests — its value is mostly the commodity day/night spread plus the ±markup, not tax arbitrage. This flips 2027.

## Architecture

Nothing runs in Grafana/Flux — the per-slot pricing and price join are too much for a datasource query, and results are cached. Two new Python entrypoints alongside `collector.py`, writing pre-computed rows

Grafana only reads and sums.

```
collector.py   raw 30s power_readings                          (exists)
prices.py      daily fetch Frank marketPricesElectricity  →  market_price   (NEW)
pricing.py     per complete day: Model 1/2                →  daily_cost     (NEW)
dashboard      read + sum daily_cost                                        (NEW)
```

**`prices.py`**

- POST GraphQL to Frank's endpoint (no auth); fetch electricity market prices for the target day(s).
- Write measurement **`market_price`**, tag `sys_sn` unnecessary (prices are system-independent — tag by nothing or a constant `market` source), one point per interval timestamped at its `from`: fields `market_price`, `market_price_tax`, `sourcing_markup`, `energy_tax`, `total`, `feed_in` (if used), `duration_s` (till – from).
- Run daily after prices are final (day-ahead prices are known the day before, so the previous complete day is always final). `--backfill FROM TO` for history (subject to how far back the API serves).

**`pricing.py`**

For each **complete** day not already processed at the current `model_version`:

1. Pull `power_readings` and `market_price` for the local day (Europe/Amsterdam boundaries, DST-aware).
2. Integrate `power × real_Δt` (trapezoidal) within each price interval, capping any single Δt (a long outage must not hold a stale power value across a slot).
3. Compute `import/export` for actual and counterfactual, price each slot, sum.
4. Write measurement **`daily_cost`**, tag `sys_sn`, timestamp = local midnight:

| Field | Meaning |
|---|---|
| `cost_model1` | with-battery cost (€) |
| `cost_model2` | no-battery cost (€) |
| `saving` | `cost_model2 – cost_model1` (signed) |
| `import_kwh_actual`, `export_kwh_actual` | grid energy, actual |
| `import_kwh_cf`, `export_kwh_cf` | grid energy, counterfactual |
| `delta_soc_percent` | SoC at 24:00 – 00:00 (borrow/bank indicator) |
| `delta_soc_kwh` | same in kWh — only if `BATTERY_CAPACITY_KWH` is configured |
| `coverage` | fraction of expected (DST-aware) samples present |
| `max_gap_s` | longest sample gap |
| `sample_count`, `span_s` | raw sample diagnostics |

| | |
|---|---|
| `balance_residual_kwh` | ∫■pv+grid+battery–load■ dt (quality) |
| `billed_cost` | actual Frank daily cost (optional, entered later for validation) |
| `model_version` | tag; schema/model version for cache invalidation |

### Complete-day / missing-data policy

- **Expected samples** computed for the *local* day (DST days are 23 h / 25 h → ~2760 / ~3000 samples at 30 s), not a fixed 2880.
- Process a day only if **coverage** ≥ **98%** *and* **max single gap** ≤ **15 min**. Scattered misses barely move an integral (real-Δt integration self-corrects); one long contiguous gap distorts a specific price slot, so it's gated separately.
- Thresholds are configurable.

### Caching / idempotency

- `pricing.py` skips days that already have a `daily_cost` row **at the current `model_version`**; bumping the version reprocesses.
- Idempotent writes (overwrite the day's point), so re-runs are safe.
- Never cache a day whose prices were still provisional (won't happen for the previous complete day, but guard `--backfill` against missing price intervals).

## Dashboard (new, read-only over `daily_cost`)

- **Table** — one row per processed day: date, Model 1, Model 2, signed saving, (optional) billed cost + Model-1 delta for validation, coverage/quality flag.
- **To-date stats** (fixed all-time range override): total saving, total days analysed, total kWh shifted, effective €/kWh the battery earned.
- **Selected-period stats** (follow the dashboard time picker): same four, summed over the range — trivial because rows are additive.
- **Daily saving bars + ΔSoC overlay** so cross-midnight borrow/bank days are visible and the day-to-day jitter is explained.

Note: "days analysed" (not "days with battery") — both models run on every clean day; there is no with/without split in the day set.

## Open items / risks

1. **Export price (a) vs (b)** — pin against a real teruglevering bill line.
2. **Sign-convention verification** — run `collector.py --once`; confirm `pv+grid+battery−load ≈ 0` on real samples before trusting euros.
3. **Frank API backfill depth** — confirm how far back `marketPricesElectricity` serves; older days may need an alternative source or may be unrecoverable.

4. **DST correctness** — day windows and price-slot alignment in Europe/Amsterdam including the 23 h / 25 h transition days.
5. **2027 saldering cliff** — tax netting on export ends; the "per-slot netting is exact" property breaks and export tax handling must change. The per-component price storage means the price side tracks automatically, but the *netting rule* in `pricing.py` will need a date-aware branch.
6. **Balance residual threshold** — choose the cutoff that flags/excludes a day.

## Assumptions (recap)

- Annual export ≤ import → net all returned energy (we have <1 year of data).
- 2026 saldering in full (tax + BTW refunded on export).
- Load and PV are identical between the two worlds (removing the battery changes nothing else in the house).
- The ~15% teruglever bonus is excluded (specific-case only).